

Relatório de Diagnóstico Técnico e Dívidas Técnicas (Etapa 1)

Grupo 35: Git Profile Search

Disciplina: F113 - Transformação Digital | UNIFOR

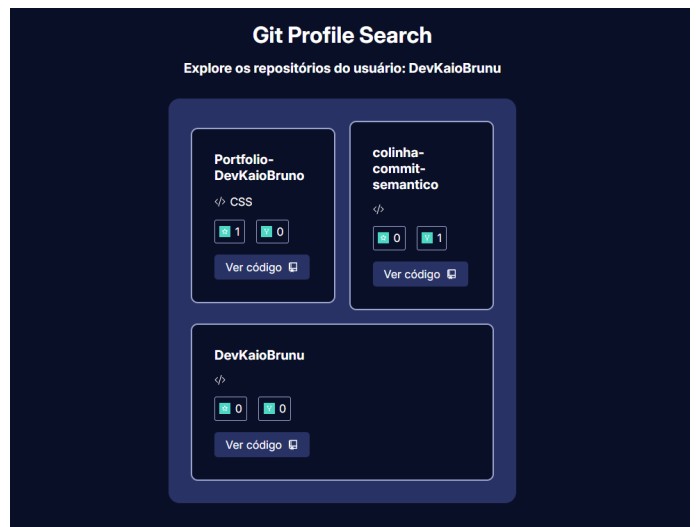
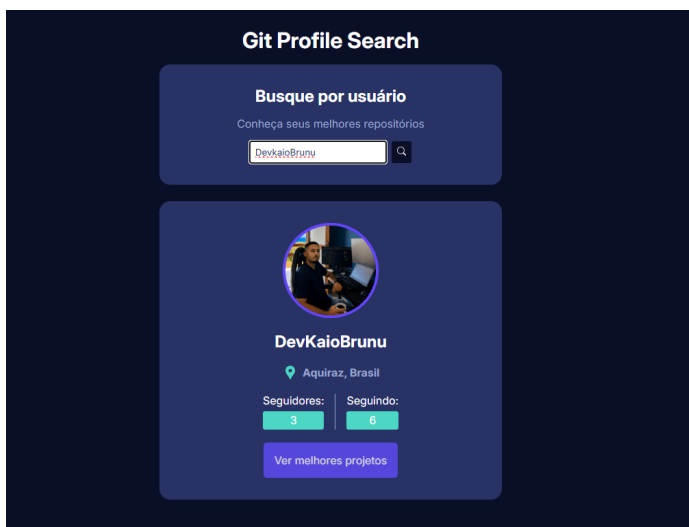
1. Introdução ao Diagnóstico

Para iniciar o ciclo de transformação do **Git Profile Search**, realizamos uma auditoria profunda no estado atual ("Baseline") da aplicação. O objetivo foi identificar gargalos que impedem a escalabilidade, comprometem a experiência do usuário (UX) e dificultam a manutenção por parte da equipe de engenharia.

2. Mapeamento do Produto e Fluxos

Antes da análise técnica, mapeamos a estrutura funcional para garantir que todos os pontos de contato do usuário fossem auditados.

- **Telas Principais:**
 - **Home/Busca:** Interface inicial com input de pesquisa.
 - **Resultado de Perfil:** Exibição de avatar, bio e seguidores.
 - **Ver melhores projetos:** Exibição dos melhores repositórios.
- **Fluxo Crítico:** Entrada de dados -> Consumo da API GitHub -> Renderização de UI.



I. Front-end (O Núcleo da Aplicação)

O front-end é onde reside toda a lógica de interface e estado.

- **Framework & Build Tool: React 18** com **Vite**. O Vite é utilizado para um *bundling* ultra-rápido via ES Modules.
- **Linguagem: TypeScript**, garantindo tipagem estática e reduzindo erros em tempo de execução (especialmente ao manipular as respostas da API).
- **Estilização: CSS Modules**, que permite escopar o CSS por componente, evitando vazamento de estilos (essencial para a refatoração do Hero e Footer).
- **Gerenciamento de Estado:** Baseado em **React Hooks** (`useState`, `useEffect`).

II. Back-end (Arquitetura de Integração)

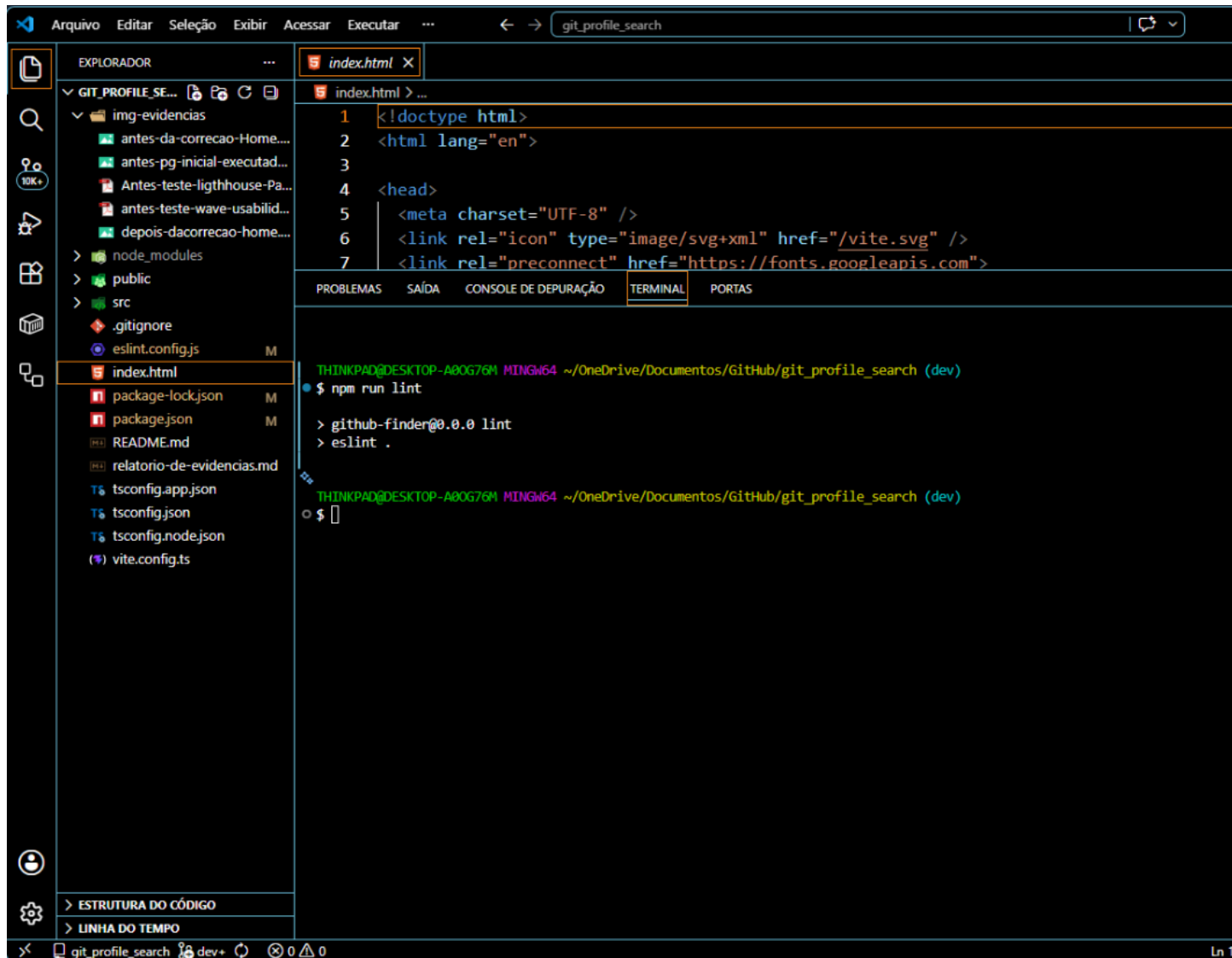
Diferente de um sistema com servidor próprio, este projeto utiliza uma arquitetura **API-First/Integrator**:

- **Provedor de Dados: GitHub REST API**. A aplicação atua como um cliente que consome os endpoints de `/users` e `/repos`.
- **Protocolo:** Comunicação via HTTPS utilizando o método `fetch` (ou Axios), processando respostas em formato **JSON**.
- **Autenticação:** Atualmente via *Public Access*, mas com estrutura para implementação de *Tokens* caso o limite de requisições seja atingido.

3. Análise de Qualidade de Código e Dívidas Técnicas

3.1. Análise Estática (ESLint)

Utilizamos o **ESLint** para verificar a conformidade com padrões modernos. Embora o resultado aponte "0 Erros", identificamos uma **falsa segurança**.



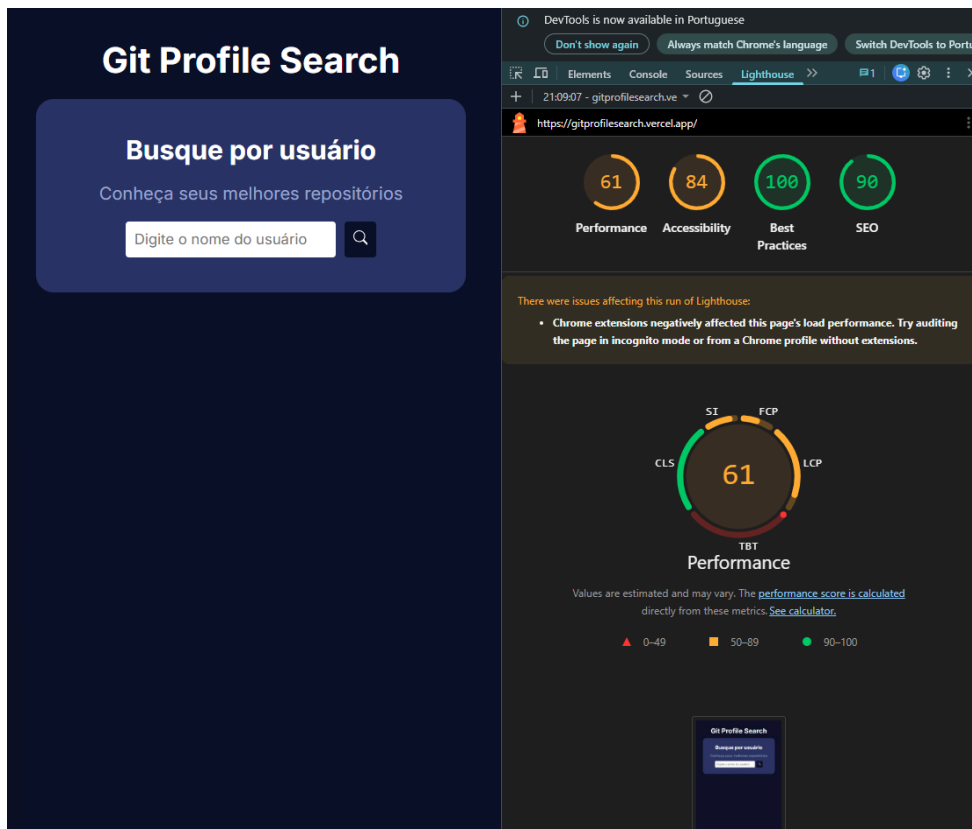
O fato de a análise estática inicial do ESLint apontar '0 Erros' gerou uma falsa sensação de conformidade e segurança na equipe de engenharia. Isso ocorre porque as configurações padrão do linter (geradas automaticamente pelo scaffolding do Vite + TypeScript) são estritamente focadas em regras de sintaxe, tipagem estática e ciclo de vida do React.

No entanto, o ESLint padrão é completamente cego para critérios de qualidade arquitetural e de experiência do usuário. Ele não valida se o código está violando o Princípio de Responsabilidade Única (SRP) do SOLID, não detecta anti-padrões de performance (como re-renderizações desnecessárias por falta de memorização) e, crucialmente, ignora as diretrizes da WCAG 2.2. Elementos sem semântica estrutural (como o uso de `<div>` no lugar de `<header>` ou `<footer>`) ou inputs de busca sem atributos `aria-label` passam pelo linter padrão sem disparar nenhum alerta. Portanto, o resultado 'zero erros' mascarava uma severa dívida técnica estrutural

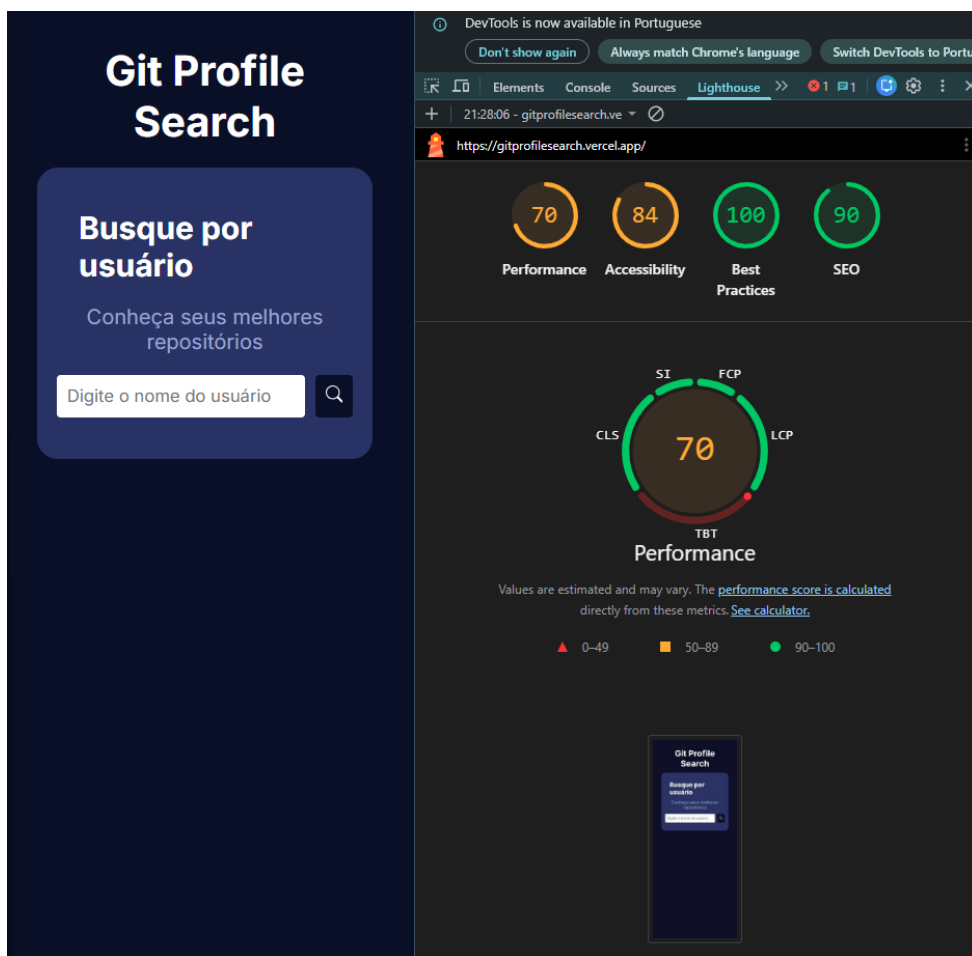
3.2. Mapeamento de Dívidas Técnicas e Code Smells

Identificamos violações de princípios de engenharia de software, especificamente o **SOLID**:

Teste Lighthouse Mobile:



Teste Lighthouse Desktop:



Relatório de teste de IA TestSprite (MCP)

Test TC001 Searches for a valid user and shows the profile details

- Test Code: TC001_Searches_for_a_valid_user_and_shows_the_profile_details.py
- Test Error:
- Test Visualization and Result:
<https://www.testsprite.com/dashboard/mcp/tests/c7499f67-3649-4746-8085-cde8bbe20c75/6a745d55-317a-40dc-9445-e703fbe45a1d>
- Status:  Passed
- Severity: LOW
- Analysis / Findings: Search flow works correctly for valid usernames. Loading state appears and the profile card displays avatar, login, location, followers, and following as expected.

Test TC002 Shows a not-found state for an unknown user

- Test Code: TC002_Shows_a_not_found_state_for_an_unknown_user.py
- Test Error:
- Test Visualization and Result:
<https://www.testsprite.com/dashboard/mcp/tests/c7499f67-3649-4746-8085-cde8bbe20c75/0e8863e3-740f-490a-8cc7-c6ad4ee09def>
- Status:  Passed
- Severity: LOW
- Analysis / Findings: The app correctly shows "Usuário não encontrado!" when the GitHub API returns 404 for a nonexistent username.

Test TC003 Searches with the keyboard and opens the profile repositories view

- Test Code:
TC003_Searches_with_the_keyboard_and_opens_the_profile_repositories_view.py
- Test Error:
- Test Visualization and Result:
<https://www.testsprite.com/dashboard/mcp/tests/c7499f67-3649-4746-8085-cde8bbe20c75/b8dc3f4c-4c7e-4c90-b772-1314493cad9>
- Status:  Passed
- Severity: LOW
- Analysis / Findings: Enter key submission and navigation to the repositories view work as intended from the profile card link.

Test TC005 Rejects an empty search submission

- Test Code: TC005_Rejects_an_empty_search_submission.py
- Test Error: Submitting an empty search did not show a validation error — the app showed a 'user not found' message instead.
- Test Visualization and Result:
<https://www.testsprite.com/dashboard/mcp/tests/c7499f67-3649-4746-8085-cde8bbe20c75/9c295e5e-0fc7-435d-997e-f92e4d5cebc6>
- Status:  Failed
- Severity: MEDIUM

- Analysis / Findings: Search.tsx does not validate empty input before calling loadUser. An empty string triggers a GitHub API request that returns 404, showing the generic "Usuário não encontrado!" message instead of a client-side validation message. Add a guard in loadUser or Search to block empty/whitespace-only submissions and display a dedicated validation message.
- Requirement: View Top Repositories
- Description: Displays the top 5 starred repositories for a user, supports direct URL access, external GitHub links, and back navigation.

Test TC004 Opens the top five repositories for a user from the profile

- Test Code: TC004_Opens_the_top_five_repositories_for_a_user_from_the_profile.py
- Test Error:
- Test Visualization and Result:
<https://www.testsprite.com/dashboard/mcp/tests/c7499f67-3649-4746-8085-cde8bbe20c75/2e8f55dd-0cce-41b5-a883-a3400a85553c>
- Status:  Passed
- Severity: LOW
- Analysis / Findings: Navigation from profile to /repos/:username works and the top 5 repositories sorted by stars are displayed correctly.
- Test TC006 Shows repository details for each top repository
- Test Code: TC006_Shows_repository_details_for_each_top_repository.py
- Test Error:
- Test Visualization and Result:
<https://www.testsprite.com/dashboard/mcp/tests/c7499f67-3649-4746-8085-cde8bbe20c75/ac3cd20d-e9c5-4883-851a-05b851e84da2>
- Status:  Passed
- Severity: LOW
- Analysis / Findings: Each repository card shows name, language, stars, and forks as expected in Repo.tsx.

Test TC007 Returns to the profile from the repositories view

- Test Code: TC007_Returns_to_the_profile_from_the_repositories_view.py
- Test Error: Returning from the repositories view did not restore the previously viewed profile. After clicking 'Voltar', the page shows the search input and heading instead of the octocat profile card.
- Test Visualization and Result:
<https://www.testsprite.com/dashboard/mcp/tests/c7499f67-3649-4746-8085-cde8bbe20c75/0bf51100-909d-4c91-b7be-7eb057a46fb4>
- Status:  Failed
- Severity: MEDIUM
- Analysis / Findings: BackBtn.tsx uses navigate(-1), which returns to / but does not preserve the user profile state (stored only in React state on Home.tsx). When navigating back, the profile is lost because state is not persisted in URL params or global state. Consider navigating to / with the username in state/query, or using a link to / that re-fetches the user.

Test TC008 Loads the repositories view directly for a username

- Test Code: TC008_Loads_the_repositories_view_directly_for_a_username.py

- Test Error:
- Test Visualization and Result:
<https://www.testsprite.com/dashboard/mcp/tests/c7499f67-3649-4746-8085-cde8bbe20c75/8a25a9ba-0d85-4bf8-ae99-3a74c721df32>
- Status:  Passed
- Severity: LOW
- Analysis / Findings: Direct navigation to /repos/:username loads and displays repositories without requiring a prior search.
- Test TC009 Opens a repository in GitHub from the repositories view
- Test Code: TC009_Opens_a_repository_in_GitHub_from_the_repositories_view.py
- Test Error:
- Test Visualization and Result:
<https://www.testsprite.com/dashboard/mcp/tests/c7499f67-3649-4746-8085-cde8bbe20c75/71535e1f-490d-4335-8184-fd329fd5a9>
- Status:  Passed
- Severity: LOW
- Analysis / Findings: The "Ver código" link opens the GitHub repository in a new tab via target="_blank" as expected.

 TestSprite
 git_profile_search

Generation Completed

Please **keep your local app (server) running** while viewing your test results and making modifications. This helps ensure everything stays in sync — thanks!

9 Tests

7

2

0

0

Pass

Failed

Blocked

In Progress

Test ID	Name	Last Execution	Created At
71535e1f	TC009 - Opens a repository in GitHub from the repositories view	 Pass 05/15/2026, 18:52	05/15/2026, 18:50
8a25a9ba	TC008 - Loads the repositories view directly for a username	 Pass 05/15/2026, 18:52	05/15/2026, 18:50
0bf51100	TC007 - Returns to the profile from the repositories view	 Failed 05/15/2026, 18:54	05/15/2026, 18:50
ac3cd20d	TC006 - Shows repository details for each top repository	 Pass 05/15/2026, 18:52	05/15/2026, 18:50
9c295e5e	TC005 - Rejects an empty search submission	 Failed 05/15/2026, 18:52	05/15/2026, 18:50
2e8f55dd	TC004 - Opens the top five repositories for a user from the profile	 Pass 05/15/2026, 18:52	05/15/2026, 18:50
b8dc3f4c	TC003 - Searches with the keyboard and opens the profile repositories view	 Pass 05/15/2026, 18:52	05/15/2026, 18:50
0e8863e3	TC002 - Shows a not-found state for an unknown user	 Pass 05/15/2026, 18:52	05/15/2026, 18:50
6a745d55	TC001 - Searches for a valid user and shows the profile details	 Pass 05/15/2026, 18:52	05/15/2026, 18:50

3 - Métricas de Cobertura e Correspondência

- 77,78% dos testes foram aprovados.

EXIGÊNCIA	TOTAL DE TESTES	APROVADO	FALHOU
PESQUISA DE USUÁRIO DO GITHUB	4	3	1
VER OS PRINCIPAIS REPOSITÓRIOS	5	4	1
TOTAL	9	7	2

4 - Principais Lacunas/Riscos

77,78% dos testes foram aprovados (7/9). Os fluxos principais de busca e listagem de repositórios funcionam bem.

Lacunas identificadas:

- 1- Validação de pesquisa vazia (TC005):** Nenhuma proteção do lado do cliente impede pesquisas vazias; os usuários veem um erro enganoso de "não encontrado" em vez da validação de entrada.
- 2- Perda do estado de navegação para trás (TC007):** `navigate(-1)` retorna à página inicial, mas os dados do perfil no estado React são perdidos, interrompendo a experiência de usuário esperada ao retornar ao perfil.
- 3- Limites de taxa da API do GitHub:** Sem isso `VITE_GITHUB_TOKEN`, buscas repetidas, automatizadas ou manuais, podem atingir os limites de taxa durante testes ou uso intenso.

Correções recomendadas:

- Adicione a validação de espaços vazios/em branco Search.tsx antes de chamar `loadUser`.
- Substitua `navigate(-1)` por uma `BackBtn.tsx` navegação explícita que preserve ou recarregue o perfil do usuário (por exemplo, estado da rota ou parâmetro de consulta com o nome de usuário).

4.2. Usabilidade (Heurísticas de Nielsen)

10 Heurísticas de Nielsen de forma explícita:

- **Heurística - 8 (Estética e Design Minimalista):** Interface excessivamente simples na Home, carecendo de uma seção Hero estruturada.
- **Heurística - 10 (Ajuda e Documentação):** Falta de orientações textuais de busca ou mensagens de erro claras ao usuário.
- **Heurística - 5 (Prevenção de Erros):** (Conexão com o teste TC005 da TestSprite) O sistema permite o envio de buscas vazias, gerando uma requisição inútil e uma mensagem enganosa de "não encontrado". Uma guard clause deve prevenir isso.
- **Heurística - 9 (Ajuda aos usuários a reconhecer, diagnosticar e recuperar-se de erros):** Mensagens de erro da API do GitHub repassadas sem tratamento adequado para o usuário final.

4.3. Performance (Web Vitals Baseline)

O diagnóstico de performance servirá como base para o comparativo "Antes x Depois" na Etapa 2.

Métrica	Valor Atual (Baseline)	Status
LCP (Largest Contentful Paint)	2.6s	Precisa Melhorar
FCP (First Contentful Paint)	2.6s	Precisa Melhorar
CLS (Cumulative Layout Shift)	0	Estável

5. Avaliação de Prontidão para Lançamento

Auditoria da infraestrutura pré-transformação:

- **Pipeline CI/CD:** Apenas deploy básico Vercel. Ausência de checks de segurança.
- **Testes:** Cobertura de 0%.
- **Estratégia de Rollback:** Manual via painel Vercel; ausência de automação.

6. Definição e Justificativa de KPIs

INDICADOR (KPI)	BASELINE (ESTADO ATUAL)	META (ETAPA 2)	JUSTIFICATIVA TÉCNICA E ORIGEM DO DADO
1. Taxa de Sucesso dos Testes Funcionais	77.78%	100%	Baseado no relatório TestSprite AI. A meta exige a correção obrigatória do bug de busca vazia (TC005) e da perda de estado no botão voltar (TC007).
2. Evitação de Spam na API (Empty Search Requests)	Alta incidência de requisições inúteis	0 requisições	Erro detectado no TC005. Mede a eficácia da implementação de uma guard clause (validação client-side) no Search.tsx para poupar a cota da API.
3. Índice de Persistência de Estado em Navegação	0% (Estado é destruído ao voltar)	100% de retenção	Falha apontada no TC007. Mede o sucesso da substituição do Maps(-1) por rotas parametrizadas por URL Query Strings para não perder o perfil buscado.
4. Maior Renderização de Conteúdo (LCP)	2.6 segundos	<= 2.0 segundo	Extraído do relatório PageSpeed Insights/Lighthouse. Avalia o ganho de performance obtido após a componentização eficiente e otimização de imagens.
5. Score de Acessibilidade Digital	84 / 100	100 / 100	Mapeado no Lighthouse e WAVE. Mede o impacto da inclusão de aria-labels nos botões de busca e correção dos 3 erros de contraste de cores.
6. Resiliência a Bloqueios de Infraestrutura (Rate Limits)	Risco crítico mapeado	0 erros HTTP 403	Alerta de risco do TestSprite. Mede o sucesso da injeção segura da variável de ambiente VITE_GITHUB_TOKEN no pipeline da Vercel.

7. Estratégia de Correção (Plano de Evolução)

Priorizamos as seguintes ações para a Etapa 2:

1. **Refatoração de UI:** Implementação de componentes semânticos (<Hero />, <Footer />).
2. **Acessibilidade:** Correção de contrastes e labels.
3. **Engenharia:** Criação de Hooks para separar lógica de API (SRP).
4. **DevOps:** Configuração de GitHub Actions para CI/CD automatizado.

